

Detailed Flutter – AI API Integration Guide

This guide explains clearly and step by step how the trained AI model is used inside the Flutter application. It describes where each part runs, how data flows, and how integration is verified.

Step 1: Where the AI Model Runs

The AI model runs on a Python server, not inside the Flutter app. The trained model is stored in the folder named 'priority_ai_model' and is loaded when the server starts.

To start the AI server, open a terminal in the AI project folder and run:

```
python priority_model_api.py
```

Step 2: How Flutter Connects to the AI Server

The Flutter application communicates with the AI server using HTTP requests. For Android Emulator testing, Flutter must use the special IP address below:

```
http://10.0.2.2:5000
```

For a physical mobile device, both the phone and the server must be on the same Wi-Fi network.

Step 3: What Data Is Sent to the AI

When a user submits a form in the Flutter app, the app sends beneficiary data as JSON to the AI API.

Example JSON sent from Flutter:

```
{ age: 25, gender: Female, income: Low, disability: No, dependents: 2, description: Single mother }
```

Step 4: How the AI Processes the Data

Inside the API, the input data is converted into a text sentence. This text is then tokenized and passed into the trained DistilBERT model. The model predicts one of three classes: Low, Medium, or High priority.

Step 5: AI Response Returned to Flutter

After prediction, the AI server sends a JSON response back to the Flutter app.

Example response:

```
{ priority: High, confidence: 0.93 }
```

Step 6: Displaying Result in Flutter

The Flutter app reads the priority and confidence values from the response and displays them to the user. This confirms successful AI integration.

Step 7: Required Flutter Configuration

Flutter must be allowed to access the internet. Add the following permission in AndroidManifest.xml:

```
<uses-permission android:name="android.permission.INTERNET"/>
```

Step 8: How to Verify Integration Works

- 1 AI server starts without errors
- 2 Flutter successfully sends a POST request
- 3 AI returns a priority value
- 4 Flutter displays the returned result

Final Conclusion

The AI model is trained offline, deployed as a REST API, and consumed by the Flutter application. This separation ensures clean architecture, scalability, and correct AI integration.